FinVeritas at FinReason Cup 2026: Exact Program Execution for Verifiable Financial Reasoning and XBRL Audit Verification

Chin-Hao Hsu
National Tsing Hua University
Hsinchu, Taiwan
c940818s@gmail.com

***Abstract*—This paper describes FinVeritas, our system for Task 1 (verifiable financial chain reasoning) and Task 3 (financial audit verification) of the FinReason Cup 2026. Language models were used throughout system development and paper preparation, but final inference for both tasks executes only explicit programs over bound inputs. For Task 1, programs over exact rationals are induced from the training questions by enumerative search, replaced by reviewed computation graphs where an induced expression fits the training data but not the stated financial identity, and complemented by routes for computations that the question states itself; the named nodes of each program are the required checkpoints. For Task 3, XBRL facts are bound by their full aspect key, and the reported and calculated values come from programs for rules 0015, 0117 and 0126 of the XBRL US Data Quality Committee, accepted under a consensus gate, with native execution of the published rules as a fallback and heuristic selection for ambiguous cases. On the Task 1 Test set FinVeritas answers 927 of the 928 questions correctly and ranks joint second of 27 teams with the same final-answer score as the top-ranked team; this includes all 348 questions from scenario families unseen in training, which are answered by programs manually specified from the unlabeled Test questions. On the Task 3 Test set it reaches 97.79% accuracy and shares first place among 23 teams; it has the lowest extraction error rate in the field and shares the best accuracy on rule 0117, the rule on which teams differ most.**

***Index Terms*—data quality rules, exact arithmetic, financial reasoning, program induction, XBRL**

I. Introduction

Exact arithmetic cannot correct a wrong input binding. A ratio with the wrong denominator, a growth factor applied to the wrong period, or an eXtensible Business Reporting Language (XBRL) total compared with a member fact from another context yields a wrong answer however precisely it is computed. The FinReason Cup 2026 [1] makes this explicit. Task 1 asks for a typed final answer and named intermediate checkpoints for multi-step financial questions [2], so a correct number reached through wrong steps is penalized. Task 3 provides an XBRL filing submitted to the U.S. Securities and Exchange Commission (SEC) and asks for two values: the value the filing reports for a target concept (*Q1*) and the value implied by the filing's own calculation or dimensional relationships (*Q2*) [3].

Both tasks separate input binding from exact deterministic execution. Each prediction is produced by an explicit program whose inputs are traceable to spans of the question or to facts of the filing, so that every checkpoint and every reported value can be inspected.

We ask three questions:

RQ1 What do reviewed computation graphs add to programs induced from the training data?

RQ2 How much of XBRL verification can published rule logic, executed exactly, decide on its own?

RQ3 Which errors remain, and do they come from the method or from the data and its evaluation?

Reviewed and induced programs disagree on 34 Test questions, 29 in the final answer; the system uses the reviewed graph in all of them. Published rule programs decide 629 of the 680 Task 3 Test cases. Fourteen of the 15 Task 3 errors occur under rule 0015; aggregate feedback suggests binding or scale issues but does not establish case-level causes.

Our contributions are:

- a Task 1 pipeline that induces exact-rational programs from training questions, replaces programs that fit the data but violate the stated financial identity with reviewed computation graphs, and emits checkpoints as nodes of the same program;
- a Task 3 pipeline that binds XBRL facts by their full aspect key, accepts rule-program results only under a consensus gate, and falls back to native execution of the published Data Quality Committee (DQC) rules;
- an analysis of both systems that attributes every Development and Test answer to the component that produced it, and a per-rule analysis of the Task 3 Test results.

II. Related Work

Financial numerical reasoning. FinQA [4], TAT-QA [5] and ConvFinQA [6] pose questions requiring arithmetic reasoning over financial reports. FinChain [7] generates questions from parameterized symbolic templates over 58 topics in 12 domains and scores both final answers and step-level consistency; the Task 1 data follow this design. Program-aided approaches such as PAL [8] and Program-of-Thoughts [9] let a language model write code that an interpreter executes. FinVeritas also separates reasoning from computation, but obtains its programs by enumerative search against labeled

TABLE I
DATA SPLITS AND THEIR USE IN THIS WORK.

Task	Split	Cases	Labels and use
1	Train	2,900	Public; program induction
1	Local dev.	290	Public; ablation (Table IV)
1	Development	580	Hidden; online aggregate score
1	Test	928	580 seen + 348 hidden; final ranking
3	Practice	332	Public (FinMR); parser and rule development
3	Development	680	Hidden; online aggregate score
3	Test	680	Hidden; final ranking

examples, in the spirit of programming by example [10], and by reviewing them against the stated financial definitions.

XBRL verification. XBRL [11] encodes each reported fact with a concept, entity, period, unit and optional dimensions, and relates facts through calculation and definition linkbases. Calculations 1.1 [12] and the duplicate-fact guidance [13] specify how summations and repeated facts are treated, and the DQC rules [14] are public consistency checks for SEC filings, executable with Arelle and its Xule processor [15]. FinAuditing [16] builds taxonomy-structured tasks from real filings under U.S. Generally Accepted Accounting Principles (US-GAAP), including the numerical task used as the Task 3 Practice set, and reports gaps in concept retrieval and cross-document reasoning. Herculean [17] finds that agent performance depends on the financial workflow, including auditing. Our Task 3 system is fully symbolic and reuses the published rule logic directly.

III. TASKS AND PROBLEM FORMULATION

Table I lists the data. In Task 1 each question specifies the answer type, unit and rounding, and the checkpoint slots to fill. The official scorer evaluates each case on its final answer (FAC) and its checkpoints, then averages hierarchically over templates, domain/topic/difficulty cells, difficulty levels, topics and domains. The Test set contains a seen stratum of 580 questions from training templates and a hidden stratum of 348 questions from scenario families that do not occur in training; the two strata are weighted equally in the final ranking [18].

Task 1 formulation. A question provides numeric literals $x = (x_0, \ldots, x_n)$, an answer specification a and a set S of checkpoint slots. A solution is a program P: a sequence of named assignments $v_k = e_k(x, v_1, \ldots, v_{k-1})$, a final node v_y and a map σ from slots to nodes. Its prediction is $\rho_a(v_y)$ for the final answer and $\rho_s(v_{\sigma(s)})$ for each $s \in S$, where ρ renders an exact value according to a specification. Checkpoints are thus intermediate nodes of the program that produces the final answer, and the task reduces to finding P and binding x to its inputs.

In Task 3 each case is one query containing the schema, the presentation, calculation, definition and label linkbases, the instance document, a US-GAAP taxonomy excerpt and the question; the query does not name the rule. Each case receives exactly one label, assigned in order: structural error, extraction error (*Q1* wrong), calculation error (*Q1* correct, *Q2* wrong) or accurate. Accuracy (ACC) is the share of accurate cases, and *Q2* is compared without tolerance. The official labels are assigned by a language model (gpt-5-mini) prompted to compare the two values by their numeric meaning [19]. Teams with equal ACC share a rank; calculation error rate determines their listing order [20]. Development and Test filings come from disjoint sets of companies. The Practice cases are labeled with the violated rule (110/120/102 cases for DQC 0015/0117/0126) and in all of them *Q1* and *Q2* differ. The task evaluates targeted numeric checks, not a full financial-statement audit.

Task 3 formulation. A filing is a set F of numeric facts, each with an aspect key and a value $\nu(f)$, and the question names a target concept q and a period t. Let $r \in \{0015, 0117, 0126\}$ be the rule the case was built around, which is not given, and $E_r(q,t) \subseteq F$ the facts of q in t to which r applies. The answer is $Q1 = \nu(f^*)$ for the fact $f^* \in E_r(q,t)$ that violates r, and $Q2 = R_r(F, f^*)$, where $R_{0015}(F,f) = |\nu(f)|$, $R_{0117}(F,f)$ is the sum of the members of f on a listed axis, and $R_{0126}(F,f)$ is the weighted sum of the calculation children of f. A system can therefore fail in three places: the choice of r, the choice of f^* when several facts are eligible, and the operand set of R_r.

IV. TASK 1: EXACT FINANCIAL PROGRAMS

Fig. 1(a) shows the pipeline. A question is answered by the first applicable route, in the order question-stated policy, explicit formula, reviewed or induced program, and scenario program. The implementation contains further fallback routes for questions that none of these routes answers; they answer no Development or Test question and are omitted here.

A. Binding and wording signatures

The question text before the answer-format block is scanned for numeric literals (integers, decimals and fractions). Each literal becomes a binding x_i, numbered in order of occurrence, with its exact rational value and character span. Replacing every literal by a placeholder gives the question's *wording signature*. The 2,900 training questions, ten per template, have 320 signatures over 290 templates.

B. Program induction

Programs are induced per signature from training value vectors. Initial atoms retain the first expression per vector; candidate-pool expansion keeps the lowest heuristic cost, then shortest expression. Atoms cost 0, unary or binary operations add 1, closed forms cost 3–7, and reused checkpoints cost 2. Initial atoms are bindings and $\{0, 1, 2, 3, 4, 12, 100, 365, 10000\}$. Search uses negation, absolute value, $+, -, \times, \div, \min, \max$, integer powers with $|t| \leq 24$, and $g = (1 + r/100)^t$, Ag, $A(g-1)$, A/g and $A(1 - 1/g)/(r/100)$ for $0 \leq t \leq 40$. Closed-form rates must vary; amounts must vary and exceed 50 in magnitude in some training example. Pool insertion rejects derived vectors exceeding magnitude 10^{100} or denominator bit length 1,024; the final

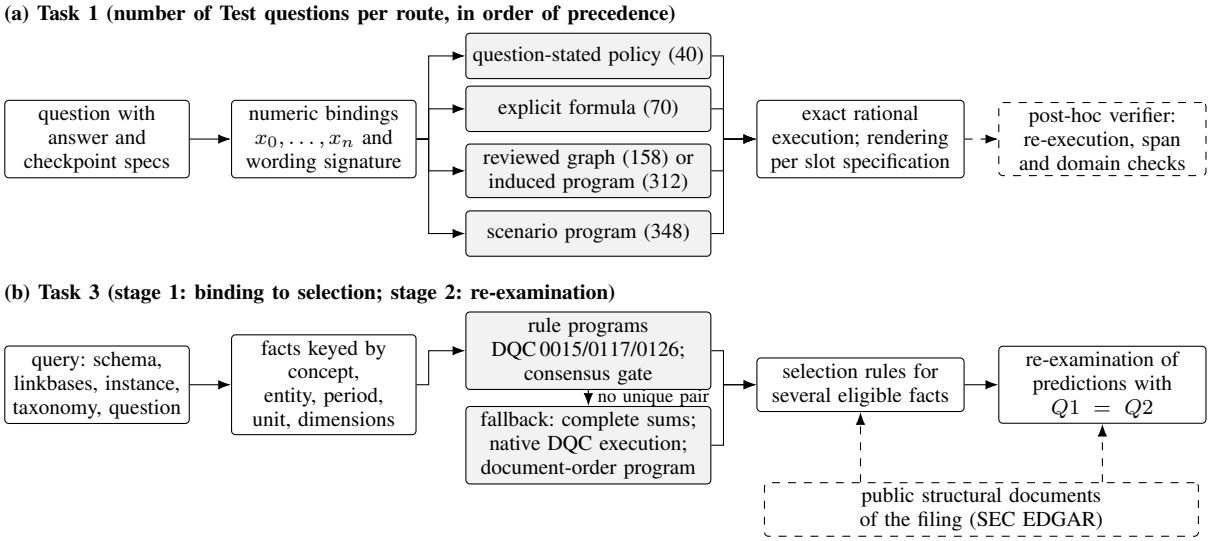


Fig. 1. Overview of the two pipelines. Shaded boxes are program sources; all arithmetic is exact. Dashed: post-hoc verification (a) and external structural documents (b, Section V-F).

binary extension uses a separate gate. Numeric checkpoints use exact-vector lookup or the first matching binary combination, joining the search for up to four rounds.

Numeric finals first use the cheapest stored candidate matching every gold output string. If none matches, one more $+, -, \times, \div$ combination of a base or checkpoint atom and a stored candidate is tried, with operand costs totaling at most 9. A first-example rounding gate precedes full-vector matching; the first match is accepted, without a global minimum-cost guarantee. Categorical finals and checkpoints seek a consistent sign-to-label map when labels vary. Algorithm 1 summarizes the numeric branch. Induction yields numeric finals for 209 signatures and categorical rules for 16; the remaining 95 are 36 policy, 21 explicit-formula and 38 other unsolved signatures, covered by later routes.

Algorithm 1 Numeric program induction for one wording signature

Require: training questions $q_1, \ldots, q_m$ of the signature with bindings $x^{(j)}$ and gold answers
Ensure: checkpoint expressions $\{e_s\}$ and a final expression e_y, or failure
$B \leftarrow$ deduplicated binding and constant atoms; $V \leftarrow$ their value vectors
expand V with unary, binary and filtered closed forms from B, keeping the cheapest expression per vector
for round $= 1, \ldots, 4$ **do**
  for each unsolved numeric checkpoint s with gold vector g_s **do**
    if $g_s \in V$ or $g_s = a \circ b$ with $a, b \in V$, $\circ \in \{+, -, \times, \div\}$ **then**
      $e_s \leftarrow$ first matching expression; add it to B and V with reuse cost 2
    end if
  end for
  stop if no checkpoint was solved; otherwise expand V using checkpoint atoms and B
end for
$e_y \leftarrow$ cheapest $v \in V$ with $\rho_a(v_j) = \rho_a(\text{gold}_j)$ for all j
if absent, take the first matching bounded binary extension described above
return $(\{e_s\}, e_y)$, or failure if e_y remains absent

C. Reviewed financial graphs

Ten training questions do not always determine the intended relationship uniquely (Section VI-B). For 79 templates we therefore transcribed the definition given in the question text, for example variants of the weighted average cost of capital (WACC), Lintner dividend smoothing or one-period binomial option pricing, into a small graph of named assignments, a final expression and a map from checkpoint slots to nodes. A reviewed graph replaces the induced program of its template whenever one exists.

D. Question-stated computations

Two question families state their computation in the text. Policy questions begin with a fixed preamble ("Apply only this fictional scenario-local …"), define thresholds and flags, and ask for one of several categorical outcomes; the policy route recognizes 20 such policies by their checkpoint identifiers and evaluates the stated comparisons with the question's numbers and flags. Explicit-formula questions begin with "Use exact arithmetic.", assign named variables and print the formula; the explicit route reads the assignments, recognizes the printed formula among 31 known forms, and evaluates the formula and its checkpoint subexpressions over the named variables. Questions of either family that a route does not recognize fall through to the reviewed or induced program. On Test the policy route answers 40 of the 100 policy questions and the explicit route 70 of the 148 explicit-formula questions.

E. Scenario programs for the hidden stratum

The 348 hidden-stratum Test questions come from 116 multi-stage scenario families of three questions each, such as stress-tested capital decisions, three-stage ratchets or comparisons of financing plans. These families do not occur in training, so the training-only induction route cannot cover them. We manually specified one program per family from the unlabeled Test question text. A program consists of a marker phrase, the expected number of numeric literals, named intermediate assignments, a final expression and one expression per checkpoint; categorical decisions are written as conditional expressions. A program applies only if its first checkpoint identifier, marker phrase, checkpoint count and literal count all match the question and no other program does; literals are bound in order of occurrence, and Boolean flags are read from fixed phrases. No answer-correctness feedback was available before submission closed; these results measure scenario transcription, not automatic generalization to unseen families.

F. Execution and rendering

All routes evaluate expressions with whitelisting interpreters over Python `Fractions`. The main interpreter accepts integer literals, bindings, the four arithmetic operations, $\min$, $\max$, absolute value, floor, ceiling, comparisons, integer powers up to 120 and geometric sums of up to 120 terms; it rejects expressions with more than 256 syntax nodes and values with more than 4,096 digits. Values stay exact until rendering, which applies each slot's specification: rounding half away from zero at the stated number of decimals, or an exact value. Rendered values are never reused, so rounding cannot propagate between checkpoints.

G. Post-hoc verification

A separate verifier checks every prediction after inference: identifiers and checkpoint sets must match the question, numeric values must be finite and categorical values must lie in their allowed sets, each binding's character span must still contain its literal, and reviewed and induced programs are re-executed with a second, independently written interpreter.

FinChain use. We used the public FinChain repository (commit `9bd2942`) only as a reference for the operator vocabulary and the closed forms of the search space. No FinChain template is matched and no FinChain code is executed at inference, so FinChain contributes to no prediction directly.

H. Example

A local-development question [2] asks for the disposal gain or loss of an asset costing USD 100,000, with zero residual value and a nine-year useful life, sold for USD 65,000 after nine months plus four full years of straight-line depreciation. The final answer requires two decimals; annual depreciation, accumulated depreciation and carrying amount are exact checkpoints. Binding gives x_0=100000, x_1=9, x_2=9, x_3=4, x_4=65000. Induction recovers only x_0/x_1; the reviewed graph supplies the rest:

$$\begin{aligned} a &= x_0/x_1 = 100000/9, \\ d &= a\,(x_2/12 + x_3) = 475000/9, \\ c &= x_0 - d = 425000/9, \\ y &= x_4 - c = 160000/9. \end{aligned}$$

The checkpoints are emitted exactly, and only the final value is rounded, to `17777.78`; all four values match the reference.

Algorithm 2 Task 3 inference for one case

Require: query with filing F, target concept q and period t
Ensure: answer pair $(Q1, Q2)$

parse F; bind facts by aspect key; normalize t
$\mathcal{P} \leftarrow$ violating programs of DQC 0015, 0117 and 0126 for the facts of q in t
if the consensus gate, Eq. (3), accepts $\mathcal{P}$ **then**
    $y \leftarrow$ the agreed pair
else if one non-dimensional target has all calculation children **then**
    $y \leftarrow$ its complete Calculations 1.1 sum
else
    $d \leftarrow$ document-order program; $n \leftarrow$ bound native finding
    $y \leftarrow n$ if n exists and is consistent with d, else d
end if
$y \leftarrow$ selection rules (Section V-D) applied to y
if *Q1* $=$ *Q2* in y **then**
    $y \leftarrow$ re-examination of y (Section V-E)
end if
return y

V. Task 3: Rule-Bound XBRL Verification

Fig. 1(b) and Algorithm 2 give the pipeline: a first stage of rule programs, fallbacks and selection rules, and a second stage that re-examines predictions with *Q1* $=$ *Q2*. Table VI separates initial answer sources from subsequent changes.

A. Parsing and fact binding

Each query is split into its sections, and every XML section is parsed with entity expansion, document type definition (DTD) loading and network access disabled. Concepts are identified by their expanded name (namespace URI and local name), never by prefix spelling. A numeric fact f is bound to the key

$$k(f) = (q_f,\, e_f,\, t_f,\, u_f,\, \delta_f), \tag{1}$$

with concept q_f, entity e_f, period t_f, unit u_f and the set δ_f of explicit and typed dimension members. Facts with equal keys are duplicates. Following [13], duplicates with equal declared decimals must have equal values, and duplicates with different decimals are consistent if their rounding intervals intersect; a consistent group is reduced to its most precise fact, and an inconsistent group blocks the programs that would use it. Nil facts are not treated as zero. If the instance document is truncated, the facts of its longest well-formed prefix are used. Periods are compared with the period equality of XBRL 2.1, in which an instant denotes the end of the day it names, after the period in the question has been normalized.

B. Rule programs and consensus gate

DQC 0015 requires that concepts on its published non-negative list, outside its axis and member exclusions, are not reported negative; for an eligible negative fact, *Q1* is the reported value and *Q2* its absolute value, following the benchmark's correction convention [20]. DQC 0117 compares a total with the sum of its members on the rule's listed axes, holding all other dimensions fixed and letting a reported subtotal replace its descendants. DQC 0126 compares a statement total with the weighted sum of its direct calculation children, which must all be present and aspect-aligned. Sums are computed over exact rationals. A DQC 0117 or 0126 program reports a violation when the reported value v and the computed value R differ beyond the rounding tolerance of the published rules,

$$|\mathrm{rd}_d(v) - \mathrm{rd}_d(R)| > m \cdot 10^{-d}, \tag{2}$$

where rd_d rounds half to even at d decimals; for DQC 0117, d is the smallest declared decimals of the total and its members, at most 5, and $m = 2$; for DQC 0126, d is the declared decimals of the total and $m = n + 1$ for n addends. Values exactly on a rounding boundary are left undecided. Since the query does not name the rule, every program applicable to a target fact is evaluated.

Let $\mathcal{P}(c)$ be the applicable programs that report a violation for case c. The consensus gate accepts a pair only when these programs agree:

$$y(c) = \begin{cases} (r, s) & \text{if } \{(r_p, s_p) : p \in \mathcal{P}(c)\} = \{(r, s)\}, \\ \bot & \text{otherwise,} \end{cases} \tag{3}$$

where $\bot$ passes the case to the fallbacks. The gate also requires that the target facts lie in a single entity and unit scope and that no duplicate group of the target is inconsistent. When DQC 0117 admits several axes, each axis is evaluated as a separate program, so a pair is accepted only if all admissible axes imply it.

C. Fallbacks

For $y(c) = \bot$, three fallbacks are tried in order. First, a unique non-dimensional target with all calculation children uses its complete Calculations 1.1 sum [12]. Second, original DQC US rule bodies [14] run with Arelle 2.46.1 and Xule [15] in a fresh offline workspace, using a reduced taxonomy and the asked period. A native finding must match the target concept, context, unit and period; its reported value must equal the sole value of that concept in that context and unit in the supplied instance. Its *Q2* is the magnitude for DQC 0015 or the message's sum for DQC 0117/0126. Each first-stage native invocation uses hash seed 0 and requires a unique bound pair, whose *Q1* must agree with the third fallback unless it has no candidate. Re-examination can invoke native execution separately; whole-pipeline repeatability is checked separately. The third fallback uses earlier rule programs, taking the first eligible negative target fact in document order and its magnitude.

D. Selection rules

Some filings report several eligible facts for the target concept, one per legal entity, plan, debt facility or segment, and the question does not say which is meant. Seventeen selection rules handle such cases. Seven are triggered by a property of the filing or target, such as an ownership axis or the concept `PaymentsForRestructuring`; the others prefer, for example, the state-and-local tax scope, a postretirement plan type, the legal-entity member whose value is uniquely negative, or the revolving facility whose value changes least across periods, and some refer to extension members of a particular issuer. The rules were developed with Development feedback. Relative to the initial-stage predictions, they change 17 Development output pairs and no Test output pair.

E. Re-examination of unchanged predictions

In every labeled Practice case *Q1* and *Q2* differ, and an extraction-only baseline that outputs $Q2 = Q1$ scores 0% on Development. A prediction with $Q2 = Q1$ therefore indicates that no violation was found, and the second stage examines such cases again. For DQC 0015 targets with eligible negative facts in the asked period, the most frequent eligible negative value (ties: smallest magnitude) is taken as *Q1* and its magnitude as *Q2*. Other cases are examined with progressively weaker evidence, stopping at the first step that yields a finding:

1) native execution with the filing's original structural documents (Section V-F) and the supplied instance;
2) native execution with the supplied documents, treating disclosure roles as statement roles, since the native DQC 0117 assertion applies to statement cubes;
3) our own DQC 0117 program on the supplied documents.

The last step tries definition-tree members first, then also admits members absent from those networks if no candidate is found. Among candidate axes it takes the last in the fixed DQC rule-axis order; its one Test change uses the broader set. A native step is accepted when it yields a single finding bound to the target fact, or several of which the DQC 0117 program selects one. No Development prediction has $Q2 = Q1$ after the first stage, so this stage changes no Development output pair; it changes 17 Test output pairs: 6 by the DQC 0015 rule and 8, 2 and 1 by the three steps.

F. External resources

Besides the case query, inference reads public DQC sources and concept lists, generic taxonomies from xbrl.org, FASB and the SEC, and a fixed development-time catalog of schema/linkbase documents from 36 SEC EDGAR filings. The catalog is not exhaustive. Documents are matched by entity, target namespace and schema reference, then hash-checked. They contain no instance documents or data-quality findings; all filing facts used to produce the answers come from the supplied instance. These structures are not shipped with the cases. Withholding them changes seven Development outputs, all among the 17 changed by selection rules, and no Test output before re-examination; step 1 uses them for eight Test outputs.

TABLE II
FACTS OF NOTESPAYABLECURRENT IN THE PRACTICE EXAMPLE.

Instant	Dimension (axis: member)	Value
2024-03-31	none	2,400,000
2024-03-31	RelatedPartyTransactions…: Shareholder	475,050
2024-03-31	RelatedPartyTransactions…: OtherAffiliates	110,500
2024-03-31	DebtInstrument: CEOandCFOAdvances	3,490
2023-03-31	none	nil
2023-03-31	RelatedPartyTransactions…: Shareholder	222,771
2023-03-31	RelatedPartyTransactions…: OtherAffiliates	nil
2023-03-31	DebtInstrument: CEOandCFOAdvances	200

G. Example

A Practice case [3], [16] of rule DQC 0117 asks for the reported and the calculated value of `NotesPayableCurrent` on 2024-03-31. Table II lists the eight facts of this concept in the instance; all share entity and unit (USD).

The asked instant removes the four 2023 facts, and the only remaining non-dimensional fact gives $Q1 = 2{,}400{,}000$. DQC 0117 lists the related-party axis but not the debt-instrument axis, so the program sums the two related-party members: $475{,}050 + 110{,}500 = 585{,}550$. With $d = 0$ and $m = 2$, the difference far exceeds the tolerance of Eq. (2), the target belongs to a balance-sheet statement cube, and no other program reports a violation, so Eq. (3) returns $(Q1, Q2) = (2{,}400{,}000,\ 585{,}550)$, which matches the reference.

VI. EXPERIMENTS

A. Setup

Both systems run offline on a single workstation with Python 3.11. Task 1 uses only the standard library; Task 3 additionally uses lxml, Arelle 2.46.1 with Xule, and the DQC US rules at revision `15523c2` (approved v29 branch). Development and Test predictions use the same code and configuration. Task 3 inference emits decimal strings; submission packaging adds thousands separators and removes trailing `.0` without changing numeric values. Local-development scores use the official scorer.

B. Task 1 results

FinVeritas scores 1.000000 FAC and 1.000000 checkpoint accuracy on the 580 Development questions. For reference, the organizers' model-based baselines score 0.980556/0.987725 (Standard) and 0.984444/0.990000 (Advanced) [18]. All 928 Test and 580 Development predictions pass the post-hoc verifier, which recomputes 1,318 values in each set.

Test. Table III reports the official Test results [21]. FinVeritas answers 927 of the 928 questions correctly and ranks joint second of 27 teams. Four teams tie on the first two ranking keys and are separated by the checkpoint score. The published Hidden FAC of 1.000000 implies that all 348 hidden-stratum questions, answered by the hand-written scenario programs, are correct, so the single wrong final answer lies in the seen stratum. The organizers report one Test question whose wording contradicts its reference answer, not yet adjudicated [21]; as the Test answers are private, we cannot tell whether it is the one we miss.

TABLE III
TASK 1 TEST RESULTS (OFFICIAL SCORER, 928 QUESTIONS, 27 TEAMS [21]). FAC AND CHECKPOINT SCORES WEIGHT THE SEEN AND HIDDEN STRATA EQUALLY; THE LAST COLUMN COUNTS CORRECT FINAL ANSWERS.

System	Comb. FAC	Hidden FAC	Checkpoint	Correct
FinVeritas (joint rank 2)	0.999167	1.000000	0.997500	927
Rank 1	0.999167	1.000000	0.999583	927
Rank 4	0.999167	1.000000	0.995417	927
Median of 27 teams	0.992341	0.989683	0.988745	921

TABLE IV
TASK 1 ROUTE ABLATION ON THE 290 LOCAL-DEVELOPMENT QUESTIONS (OFFICIAL SCORER). EACH ROW ADDS ONE ROUTE.

Configuration	Finals	Slots	FAC	Checkpoint
Induced programs	218	449/518	0.7496	0.9020
+ reviewed graphs	270	501/518	0.9419	0.9754
+ question-stated policy	281	509/518	0.9786	0.9893
+ explicit formula	290	518/518	1.0000	1.0000

Table IV adds routes cumulatively on the 290 local-development questions. Induced programs alone answer 218 questions, all correctly; the remaining questions fail because no complete final program was found, not because a program computes a wrong value. Reviewed graphs add 52 correct answers, the largest single gain, and the two question-stated routes complete the set.

Where reviewed graphs matter. Reviewed graphs and induced programs both produce an answer for 27 local-development, 54 Development and 54 Test questions. They agree on all local-development and Development questions but disagree on 34 Test questions from 19 templates, 29 of them in the final answer. In 12 of the 34, a parameter that was constant across the ten training questions of the template takes a new value on Test, and the induced program had absorbed it into a literal; in the other 22, the induced expression agrees with the stated formula only on the training values. For example, a cross-currency question asks for euros $\times$ (USD per EUR)/(USD per GBP). In all ten training questions the first rate is twice the second, so induction returns euros $\times 2$; a Test question quotes 1 and 2 USD, for which the stated cross rate $1/2$ gives 197 pounds, whereas the induced program would give 788. Since only one submitted Test final answer is wrong, at least 28 of the 29 reviewed final answers that differ from the induced ones are correct. This bound concerns the reviewed answers; hidden references prevent an independently scored Test ablation.

C. Task 3 Test results

Table V reports the final Test results under the official judge. FinVeritas answers 665 of the 680 cases correctly and shares first place among 23 teams under the organizers' published ranking [20]. Our extraction error rate is the lowest in the

TABLE V
TASK 3 TEST RESULTS (OFFICIAL JUDGE, 680 CASES, 23 TEAMS [20]). E, C: EXTRACTION AND CALCULATION ERROR RATES; THE LAST THREE COLUMNS ARE ACC WITHIN EACH RULE (162/54/464 CASES). ALL VALUES IN %.

System	ACC	E	C	0015	0117	0126
FinVeritas (joint rank 1)	97.79	0.29	1.91	91.36	98.15	100.00
Median of 23 teams	95.88	1.18	2.65	90.12	83.33	100.00
Best per column	97.79	0.29	1.32	91.98	98.15	100.00

TABLE VI
TASK 3 INITIAL ANSWER SOURCES AND SUBSEQUENT CHANGES.

Component	Development	Test
Rule programs, consensus gate	618	629
Complete Calculations 1.1 sum	25	9
Native DQC finding	15	10
Document-order program	22	32
of these, changed by selection rules	17	0
of these, changed by re-examination	0	17

field, and on DQC 0117, where team accuracies range from 5.56% to 98.15%, FinVeritas shares the best result.

Where the Test cases are lost. Official per-rule aggregates place 14 of the 15 errors in DQC 0015 (148/162 correct). Only 2 cases overall are extraction errors, so at least 12 of the 14 are labeled calculation errors. Our output has exactly 162 pairs in the DQC 0015 form (listed non-negative target, $Q1 < 0$, $Q2 = -Q1$), matching the official rule count, but this does not identify their hidden case labels. If these sets coincide, a discrepancy in the bound reported value or its scale could explain a failed negation despite an accepted *Q1*. This is a hypothesis, not a case-level diagnosis. Six of our sign-form predictions come from re-examination (Section V-E); their individual correctness is unknown. No team exceeds 91.98% on DQC 0015, and the organizers also highlight reported-value and scale selection [20].

D. Sources of the Task 3 answers

Table VI counts the initial answer sources before selection and re-examination, then the cases changed by those steps. Native findings supply 15 Development and 10 Test initial pairs; later re-examination adds native-derived changes. Native execution also informed development: our programs follow conditions from the published Xule sources, including statement cubes, listed axes, concept names and period equality, and were checked against native results.

Agreement with the reference implementation. Table VII compares the final answers with the native finding bound to the target fact, obtained by executing the published rules on every case. Where a single native finding is bound, our answer equals it in every case except two Development cases, which the judges accept; where several are bound, the selection step decides. The 12 Test cases without a bound finding are 4 dimensionally invalid targets, 4 filings for which the rules report nothing and 4 in which they report findings for other facts only. For unique DQC 0117 and 0126 matches the operand sets also coincide (474 Development and 487 Test cases), so the answers agree with the reference rules not only in value but in the facts summed.

TABLE VII
FINAL TASK 3 ANSWERS COMPARED WITH THE BOUND NATIVE DQC FINDING.

Relation to native finding	Development	Test
Equal to the unique bound finding	654	646
One of several bound findings	20	22
Different from the bound finding	2	0
No bound finding	4	12

E. Determinism and verification

Rerunning both systems from source reproduces every submitted Task 1 record and every Task 3 value, and two Test runs of Task 3 are byte-identical. Across Development and Test, all 1,360 *Q1* values occur as facts of the asked concept and period in the supplied instance. An audit re-derives 787 candidate-program operand-witness sums with exact rationals (391 Development, 396 Test), with no arithmetic failures. These checks do not certify the intended context or full rule applicability, and do not independently recompute every calculated value.

Code and reproduction instructions: https://github.com/cs113062273-bot/IEEE-FinReason-FinVeritas.

VII. DISCUSSION

RQ1: reviewed graphs address ambiguous induction. Where both routes produce an answer, they agree on training, local-development and Development questions but diverge on new Test values. At least 28 of the 29 differing reviewed final answers are correct. Most disagreements are coincidental fits rather than absorbed constants, so checking constant parameters alone would miss them. Definition-based review addresses this ambiguity without guaranteeing correctness for arbitrary new values.

RQ2: published rule logic decides most cases. Rule programs written from the published definitions decide about nine in ten cases under the consensus gate; native execution is used as a fallback. The programs cover conditions such as the listed axes and statement cubes of DQC 0117, whose Test accuracies vary most across teams. The leaderboard alone does not isolate each condition's contribution.

RQ3: binding and evaluation remain unresolved. Fourteen of the 15 Test errors occur under DQC 0015, whose *Q2* is fixed once *Q1* is chosen. Together with the Development cases requiring selection among eligible facts, this motivates examining f^* and its scale. Aggregate Test scores cannot establish each error's cause or exclude implementation and judge errors.

Negative result: language-model selection. We also tested whether a frozen language model could replace the selection rules. Qwen3-14B (4-bit quantized, no fine-tuning), run locally, chose among the remaining candidate programs while

all arithmetic stayed deterministic; each candidate list was presented in forward and reversed order. On the 332 Practice cases the model changed at most seven answers, with a net gain of at most one and only in the reversed order, and a variant that changes an answer only when both orders agree changed none. Its choices were not stable under candidate order, so it is not part of the submitted system.

Limitations. Task 1 coverage is tied to known question families; the 116 hidden-stratum families required programs manually specified from Test text. They answer all 348 questions correctly but require new programs for new families. In Task 3, some selection rules use Development feedback and some are issuer-specific; relative to initial-stage predictions, they change 17 Development output pairs but no Test output pair, so we use blind Test results to assess transfer to new filings. Re-examination is heuristic; seven Development and eight Test answers depend on structural documents not shipped with the cases; and native execution uses a reduced taxonomy set restricted to the asked period.

VIII. Conclusion

FinVeritas executes explicit programs over bound financial inputs, with no language model invoked during final inference. It answers 927 of 928 Task 1 Test questions correctly (joint second of 27 teams) and reaches 97.79% on Task 3 (joint first of 23), tying the top primary score in both tasks. Reviewed graphs address relationships underdetermined by training values, and published rule programs decide most XBRL cases. The hidden-stratum Task 1 results depend on manually specified Test scenario programs, while Task 3 relies on disclosed XBRL heuristics.

Acknowledgment

The author thanks the FinReason Cup organizers. This independent work received no external funding. AI disclosure: OpenAI Codex and Anthropic Claude were used throughout system development and paper preparation. Claude drafted all sections from the code, results and analyses and edited the language. Codex was used for code implementation, debugging, technical revisions of Sections I–VIII, and reference and reproducibility checks. The author is responsible for the final text, claims and results.

References

[1] Z. Xie *et al.*, "Overview of the FinReason Cup 2026: Agentic financial reasoning, hedging, and audit verification," 2026, manuscript in preparation. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/

[2] ——, "Overview of the FinReason Cup 2026 task 1: Verifiable financial chain reasoning," 2026, manuscript in preparation. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/task1/

[3] Y. Wang *et al.*, "Overview of the FinReason Cup 2026 task 3: Financial audit verification," in *2026 IEEE International Conference on Big Data (BigData)*, Phoenix, Arizona, USA, 2026, manuscript in preparation. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/task3/

[4] Z. Chen *et al.*, "FinQA: A dataset of numerical reasoning over financial data," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 3697–3711. [Online]. Available: https://aclanthology.org/2021.emnlp-main.300/

[5] F. Zhu *et al.*, "TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, 2021, pp. 3277–3287.

[6] Z. Chen, S. Li, C. Smiley, Z. Ma, S. Shah, and W. Y. Wang, "ConvFinQA: Exploring the chain of numerical reasoning in conversational finance question answering," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 6279–6292.

[7] Z. Xie *et al.*, "FinChain: A symbolic benchmark for verifiable chain-of-thought financial reasoning," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, M. Liakata, V. P. Moreira, J. Zhang, and D. Jurgens, Eds. San Diego, California, United States: Association for Computational Linguistics, Jul. 2026, pp. 14 529–14 553. [Online]. Available: https://aclanthology.org/2026.acl-long.662/

[8] L. Gao *et al.*, "PAL: Program-aided language models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 10 764–10 799.

[9] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Transactions on Machine Learning Research*, 2023.

[10] S. Gulwani, "Automating string processing in spreadsheets using input-output examples," in *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. ACM, 2011, pp. 317–330.

[11] XBRL International, "Extensible business reporting language (XBRL) 2.1," 2013, recommendation of 31 December 2003 with errata corrections to 20 February 2013. [Online]. Available: https://www.xbrl.org/Specification/XBRL-2.1/REC-2003-12-31/XBRL-2.1-REC-2003-12-31+corrected-errata-2013-02-20.html

[12] ——, "Calculations 1.1," 2023, recommendation of 22 February 2023 with corrected errata of 14 February 2024. [Online]. Available: https://www.xbrl.org/Specification/calculation-1.1/REC-2023-02-22+corrected-errata-2024-02-14/calculation-1.1-REC-2023-02-22+corrected-errata-2024-02-14.html

[13] ——, "Handling duplicate facts in XBRL and Inline XBRL 1.0," 2018, Working Group Note, 19 April 2018. [Online]. Available: https://www.xbrl.org/WGN/xbrl-duplicates/WGN-2018-04-19/xbrl-duplicates-WGN-2018-04-19.html

[14] XBRL US Data Quality Committee, "DQC US rules, approved v29 ruleset branch," 2026, accessed 13 October 2026. [Online]. Available: https://github.com/DataQualityCommittee/dqc_us_rules/tree/15523c2

[15] Arelle project, "Arelle: open-source XBRL platform," version 2.46.1 with the Xule rule processor plugin was used. [Online]. Available: https://arelle.org/

[16] Y. Wang *et al.*, "FinAuditing: A financial taxonomy-structured multi-document benchmark for evaluating LLMs," in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Melbourne, Australia: Association for Computing Machinery, 2026, pp. 3456–3463. [Online]. Available: https://doi.org/10.1145/3805712.3808578

[17] X. Peng *et al.*, "Herculean: An agentic benchmark for financial intelligence," arXiv:2605.14355, 2026. [Online]. Available: https://arxiv.org/abs/2605.14355

[18] FinReason Cup organizers, "FinReason Cup: Terms of participation," 2026, accessed 24 October 2026. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/terms/

[19] ——, "FinReason Cup task 3: Submission," 2026, accessed 24 October 2026. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/task3/submit/

[20] ——, "FinReason Cup task 3: Results," 2026, accessed 29 October 2026. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/task3/leaderboard/

[21] ——, "FinReason Cup task 1: Test results," 2026, accessed 28 October 2026. [Online]. Available: https://the-finai.github.io/IEEE-bigdata-cup/task1/leaderboard/test/